

SmartHome

Controller Dashboard

• Live

USERNAME

admin

PASSWORD

Sign In

Devices

Monitor and control your connected devices

Add Device

Devices

Activity Log

3

TOTAL

3

ONLINE

0

OFFLINE

CAMERA

camera_1

inactive

Toggle State

LIGHT

light_1

off

Toggle State

FAN

fan_1

off

Toggle State

SmartHome

Devices

Add Device

Devices

3

3

0

camera_1

motion

Toggle State

light_1

on

Toggle State

Register Device

Manually commission a new device.

DEVICE ID

living_room_light

DEVICE TYPE

lig

INITIAL STATE

off

Register

Sign Out

Devices

Monitor and control your connected devices

Add Device

Devices

Activity Log

4

TOTAL

4

ONLINE

0

OFFLINE

CAMERA

camera_1

inactive

Toggle State

LIGHT

light_1

off

Toggle State

FAN

fan_1

off

Toggle State

LIGHT

living_room_light

on

Toggle State

1. Comm. Arch, TCP Socket lifecycle & Concurrency Model.
- a. System overview

Smart home controller connects 3 types of participants over single TCP based websocket server.

Web Browser → websocket (ws://)

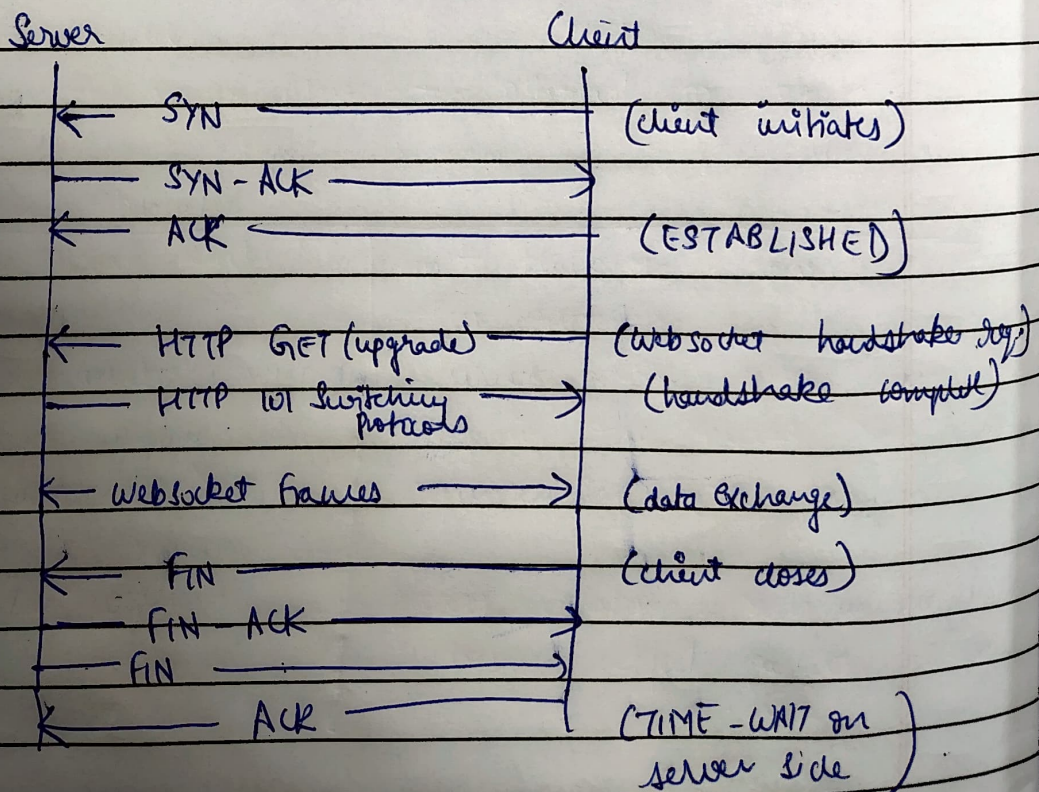
IoT device 1 → websocket (ws://)

IoT device 2 → websocket (ws://)

[c, websocket server : 8080]

- ~~websocket~~ clients send login, get-device, command, and get-log msg. [SquidB]
- IoT device clients send register, heartbeat & state-update msg. [shared memory]
- server routes command from web clients to the correct device child process using Posix shared memory + named semaphores + SIGUSER1 signal

- b. TCP socket lifecycle.



TCP socket opt used in server.

- `SO_REUSEADDR` → used in server socket → allows rebinding port 8080 even if it is in TIME-WAIT state after a restart
- `TCP_NODELAY` → each accepted client socket → disable Nagle's algorithm → commands are sent immediately w/o buffer to reduce latency
- `SO_KEEPALIVE` → each accepted client socket → OS sends keepalive probes to detect dead connection, & 'clean up' close wait states

code snippet (src/main.c)

`int opt = 1`

`setsockopt (server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));`

`int flag = 1;`

`setsockopt (client_fd, IPPROTO_TCP, TCP_NODELAY, &flag, sizeof(flag));`

`setsockopt (client_fd, SOL_SOCKET, SO_KEEPALIVE, &flag, sizeof(flag));`

websocket handshake mechanism (RFC 6455)

→ websocket begins as an HTTP/1.1 upgrade req., full RFC 6455 handshake implementation.

Send HTTP req by client → GET / HTTP/1.1

Host: local host: 8080

Server computes accept key

upgrade: websocket

connection: Upgrade

key = Base64

(SHA-1(client key +

"258EAF6... - (SHA-Bit 4)"))

Secr-WebSocket-Key: dHMi...

Sec-WebSocket-ver: 13

server replies with HTTP 101:

```
deep@deep-ASUS-TUF-Gaming-A15-FA506QM-FA506QM:~/Videos/Sem-6-Labs/NPACN_Fisac$ python3 simulator/device sim.py --port 8080
t.join()
File "/usr/lib/python3.10/threading.py", line 1096, in join
    self._wait_for_tstate_lock()
File "/usr/lib/python3.10/threading.py", line 1116, in _wait_for_tstate_lock
    if lock.acquire(block, timeout):
KeyboardInterrupt
^CException ignored in: <module 'threading' from '/usr/lib/python3.10/threading.py'>
Traceback (most recent call last):
  File "/usr/lib/python3.10/threading.py", line 1567, in _shutdown
    lock.acquire()
KeyboardInterrupt:

• deep@deep-ASUS-TUF-Gaming-A15-FA506QM-FA506QM:~/Videos/Sem-6-Labs/NPACN_Fisac$ netstat -ant | grep 8080
tcp        0      0 127.0.0.1:47856      127.0.0.1:8080      TIME_WAIT
tcp        0      0 127.0.0.1:8080      127.0.0.1:52884     ESTABLISHED
tcp        0      0 127.0.0.1:52884     127.0.0.1:8080     ESTABLISHED
tcp        0      0 127.0.0.1:47866     127.0.0.1:8080     TIME_WAIT
tcp        0      0 127.0.0.1:47840     127.0.0.1:8080     TIME_WAIT
deep@deep-ASUS-TUF-Gaming-A15-FA506QM-FA506QM:~/Videos/Sem-6-Labs/NPACN_Fisac$
```

 bash

 bash server



HTTP/1.1 101 switching protocol.

upgrade: websocket

connection: upgrade

sec-websocket-accept: a7Bd...

After this exchange, TCP connection is upgraded, both sides using websocket framing protocol instead of HTTP.

code ref (src/websocket.c)

SHA1 (unsigned char*) computes sha1 (connection), sha1 - vs
 char* key = base64_encode (sha1 - vs, 20)

sprintf (response, size of response),

"HTTP/1.1 101 switching protocol\r\n"

"upgrade: websocket\r\n"

"connection: upgrade\r\n"

"sec-websocket-accept: %s\r\n", accept

d. Concurrency model

• Use fork() per connection + pthread for bg tasks.

fork() and its justification

→ single threaded client()

can't block on recv() w/o starving other connections.

→ thread/connection (only pthread)

shared address space means mem bug in one thread crash everything significantly more complex.

→ async io (epoll)

→ fork() per connection

process isolation; crash in one child doesn't affect parent or other connections.

Filter URLs

Status	Method	Domain	File	Initiator	Type	Transferred	Size
304	GET	127.0.0.1:3000	/	document	html	cached	5.19 kB
200	GET	fonts.googleapis.com	css2?family=Inter:wght@300;400;500;600;700&display=swap	stylesheet	css	cached	-1 B
200	GET	127.0.0.1:3000	style.css	stylesheet	css	cached	6.33 kB
304	GET	127.0.0.1:3000	app.js	script	js	cached	8.93 kB
200	GET	127.0.0.1:3000	favicon.ico	img	html	cached	469 B
101	GET	127.0.0.1:8080	/	app.js:35 (websocket)	plain	129 B	0 B

|| + 🔍

AllHTMLCSSJSXHRFontsImagesMediaWSOther

☐ Disable CacheNo Throttling ⚙

📄 HeadersCookiesRequestResponseTimingsStack Trace

Filter Headers

▶ GET ws://127.0.0.1:8080/

Status101 Switching Protocols ⓘVersionHTTP/1.1Transferred129 B (0 B size)DNS ResolutionSystem

▼ Response Headers (129 B)Raw

HTTP/1.1 101 Switching ProtocolsUpgrade: websocketConnection: UpgradeSec-WebSocket-Accept: xkDMF2STMfLhgi8ZboPYHNPloc4=

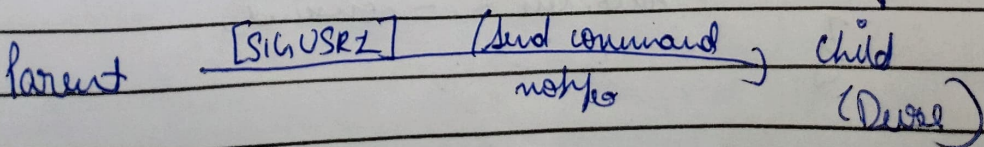
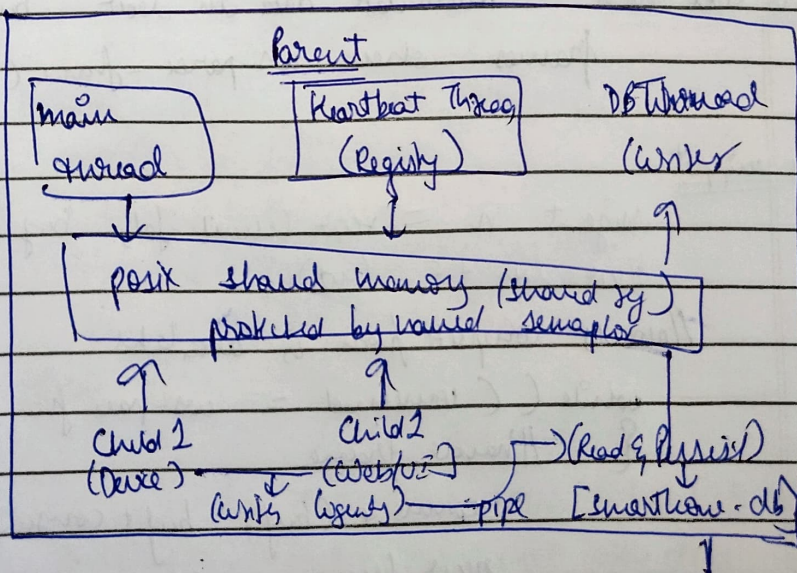
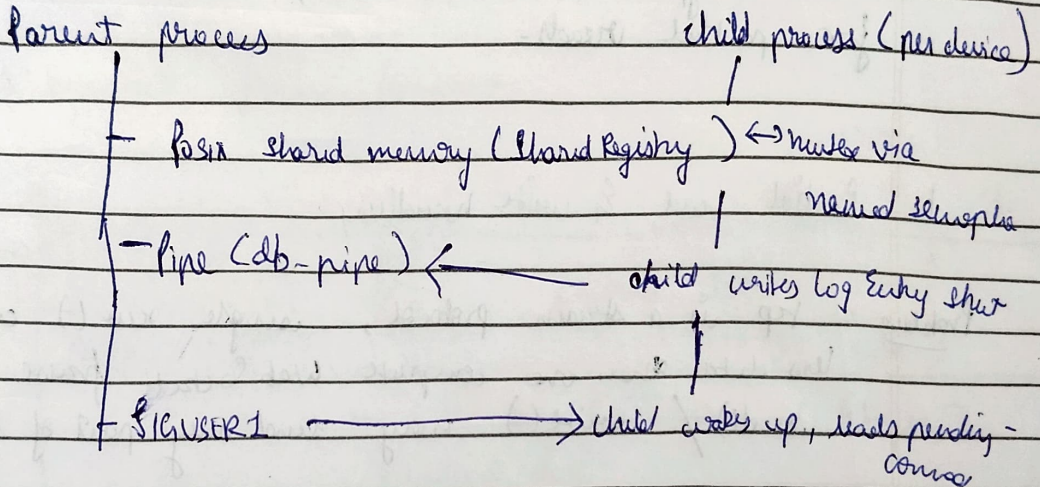
▼ Request Headers (530 B)Raw

GET / HTTP/1.1Host: 127.0.0.1:8080User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:148.0) Gecko/20100101 Firefox/148.0Accept: */*Accept-Language: en-US,en;q=0.9Accept-Encoding: gzip, deflate, br, zstdSec-WebSocket-Version: 13Origin: http://127.0.0.1:3000Sec-WebSocket-Extensions: permessage-deflateSec-WebSocket-Key: liss8sKb+7oB6FWi7kKx1g==Connection: UpgradeSec-Fetch-Dest: emptySec-Fetch-Mode: websocketSec-Fetch-Site: same-sitePragma: no-cacheCache-Control: no-cacheUpgrade: websocket

🕒 6 requests20.92 kB / 129 B transferredFinish: 114 msDOMContentLoaded: 69 msload: 70 ms

Thread Responsibility

- main thread → accept loop → for new TCP connections, fork child / client.
- db-write-thread → DB worker → drain log pipe to write to split - serializes all log work.
- heartbeat-thread → heartbeat → every 30s, scans shared memory & marks down offset if no heartbeat.

IPC b/w parent & children

Status	Method	Domain	File	Initiator	Type	Transferred	Size	0 ms	10.24 s	20.48 s	30.72 s
304	GET	127.0.0.1:3000	/	document	html	cached	5.19 kB	5 ms			
200	GET	fonts.googleapis.com	css2?family=Inter:wght@300;400;500;600;700&display=swap	stylesheet	css	cached	-1 B	0 ms			
200	GET	127.0.0.1:3000	style.css	stylesheet	css	cached	6.33 kB	0 ms			
304	GET	127.0.0.1:3000	app.js	script	js	cached	8.93 kB	1 ms			
	GET	127.0.0.1:8080	/	app.js:35 (websocket)		NS_ERROR_CONNECTION_REFUSED	0 B	0 ms			
	GET	127.0.0.1:3000	favicon.ico	img	html	cached	469 B	0 ms			
	GET	127.0.0.1:8080	/	app.js:35 (websocket)		NS_ERROR_CONNECTION_REFUSED	0 B	0 ms			
	GET	127.0.0.1:8080	/	app.js:35 (websocket)		NS_ERROR_CONNECTION_REFUSED	0 B	0 ms			
	GET	127.0.0.1:8080	/	app.js:35 (websocket)		NS_ERROR_CONNECTION_REFUSED	0 B	0 ms			
	GET	127.0.0.1:8080	/	app.js:35 (websocket)		NS_ERROR_CONNECTION_REFUSED	0 B	0 ms			
	GET	127.0.0.1:8080	/	app.js:35 (websocket)		NS_ERROR_CONNECTION_REFUSED	0 B	0 ms			
	GET	127.0.0.1:8080	/	app.js:35 (websocket)		NS_ERROR_CONNECTION_REFUSED	0 B	0 ms			
101	GET	127.0.0.1:8080	/	app.js:35 (websocket)	plain	129 B	0 B			1 ms	

2. Websocket server implementation details.

- a) handling multiple client types
↳ done through 1st msg.

TCP connect → WS handshake → first msg.

type = auth → client-web (browser / clipboard)
type = register → client-device (IoT device)

Each child process holds a 'connection' struct that tracks the client's type, state, session token, device ID and a recv-buffer for partial reads.

- b. Partial read & write handling.

Problem = TCP is a stream protocol, single `recv()` call may return less data than one complete Websocket frame.
`send()` / `write()` may send only part of frame.

Partial Read-solve - Accumulate data in `recv_buf` and only process frames when `recv-parse-frame()` returns true value.

code snippet

```

ssize_t n = recv(client_fd, buf + recv_len, size_of(buf) - recv_len, 0);
recv_len += (int)n;

// process complete frame is available
while (consumed = recv-parse-frame(buf, recv_len, ...)) {
    // handle frame
    memmove(buf, buf + consumed, recv_len - consumed);
    recv_len -= consumed;
}
    
```


Partial writes

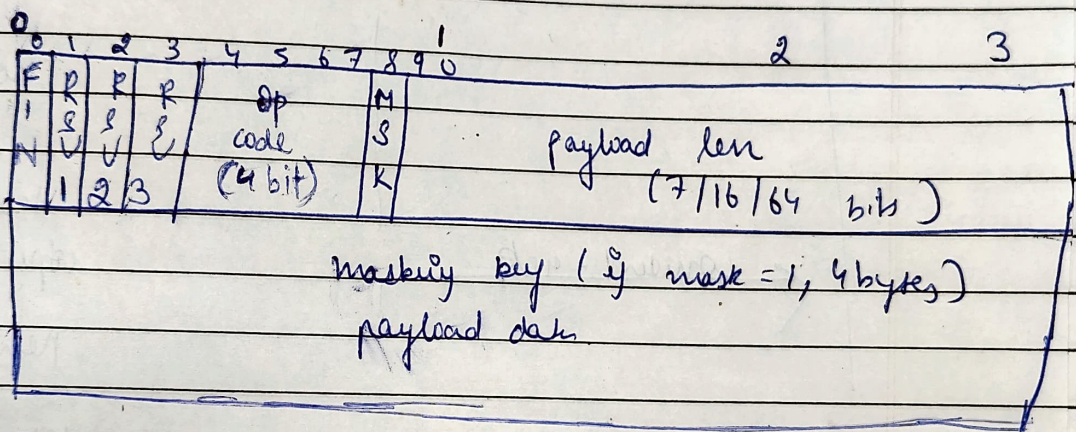
Date _____
Page _____

code snippet

as_send_frame() loops until all bytes are written.

```
while (total < frame_len) {
    size_t n = write(fd, frame + total, frame_len - total);
    if (n < 0 && errno == (EPIPE || EINTR)) {
        usleep(1000); continue;
    }
    total += (int)n;
}
```

6. WebSocket frame format.



frame parser handles all 3 payload len encoding:

- <126 → 7 bit len in byte 1
- =126 → next 2 bytes give len
- =127 → next 8 bytes give length.

opcode handled:

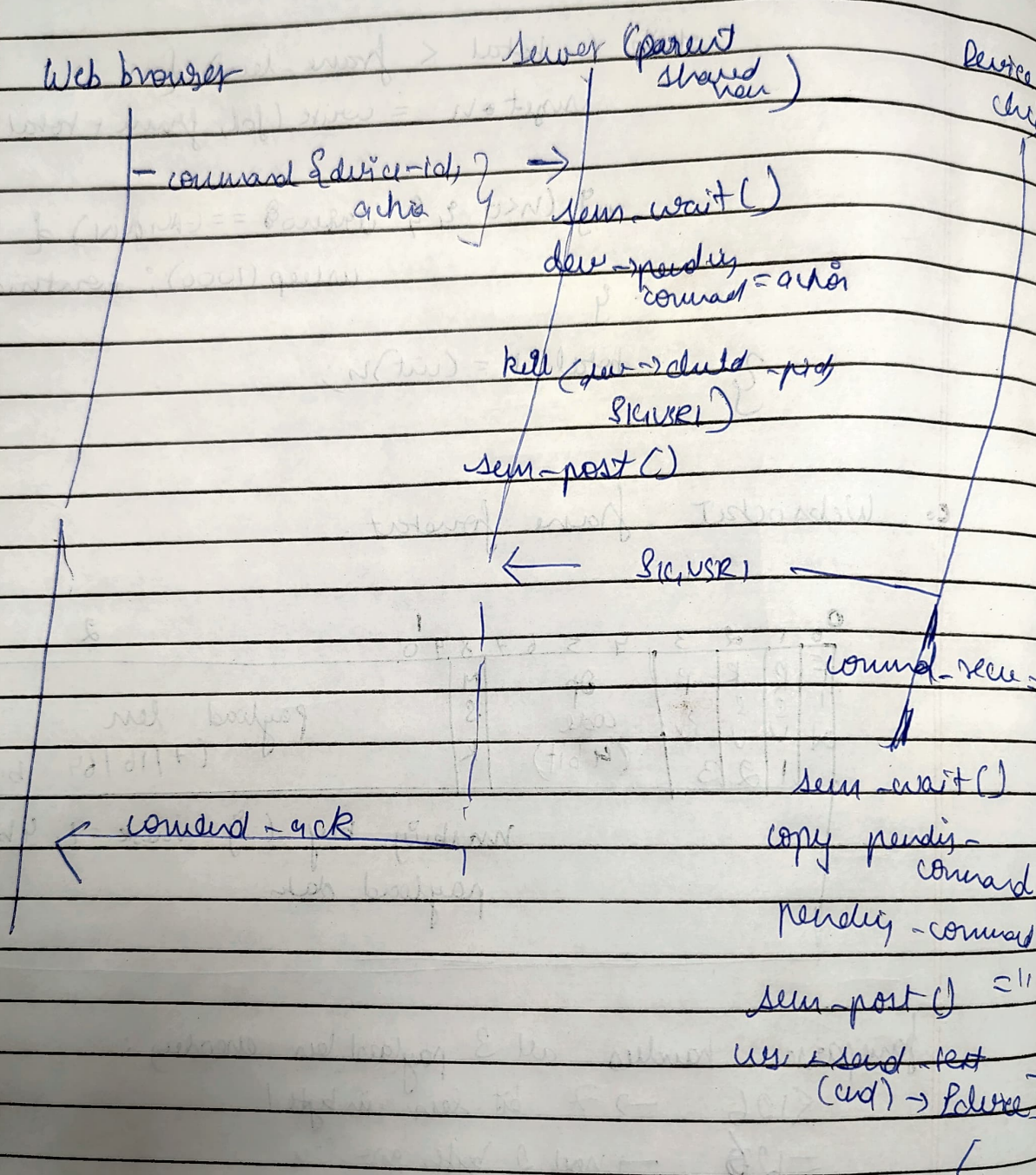
opcode	Hex	Action
text frame	0x1	route to hand_message()
binary frame	0x2	object with close code 1003
close	0x8	acknowledge frame, exit child
ping	0x9	respond with pong (same payload)
pong	0xA	send give
reserved (>0x4)	-	close with code 1009


```
deep@deep-ASUS-TUF-Gaming-A15-FA506QM-FA506QM:~/Videos/Sem-6-Labs/NPACN_Flsac$ cd client
deep@deep-ASUS-TUF-Gaming-A15-FA506QM-FA506QM:~/Videos/Sem-6-Labs/NPACN_Flsac/client$ python3 -m http.server 3000
Serving HTTP on 0.0.0.0 port 3000 (http://0.0.0.0:3000/) ...
127.0.0.1 - - [31/Mar/2026 16:41:44] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [31/Mar/2026 16:41:44] "GET /style.css HTTP/1.1" 200 -
127.0.0.1 - - [31/Mar/2026 16:41:44] "GET /app.js HTTP/1.1" 200 -
127.0.0.1 - - [31/Mar/2026 16:41:44] code 404, message File not found
127.0.0.1 - - [31/Mar/2026 16:41:44] "GET /favicon.ico HTTP/1.1" 404 -
```

```
deep@deep-ASUS-TUF-Gaming-A15-FA506QM-FA506QM:~/Videos/Sem-6-Labs/NPACN_Fisac/server$ ./server 8080
[INFO] Database initialized
[INFO] Server listening on port 8080 using database smarthome.db
[INFO] Child 23878 handling connection from 127.0.0.1:36286
[INFO] WebSocket handshake successful in child
[INFO] Auth successful
[INFO] Child 23910 handling connection from 127.0.0.1:51412
[INFO] Child 23911 handling connection from 127.0.0.1:51414
[INFO] Child 23912 handling connection from 127.0.0.1:51418
[INFO] WebSocket handshake successful in child
[INFO] WebSocket handshake successful in child
[INFO] WebSocket handshake successful in child
[INFO] Device registered
[INFO] Device registered
[INFO] Device registered
[INFO] Child 24266 handling connection from 127.0.0.1:50758
[INFO] Child 24267 handling connection from 127.0.0.1:50772
[INFO] Child 24268 handling connection from 127.0.0.1:50788
[INFO] WebSocket handshake successful in child
[INFO] WebSocket handshake successful in child
[INFO] WebSocket handshake successful in child
[INFO] Device registered
[INFO] Device registered
[INFO] Device registered
[INFO] Child 24689 handling connection from 127.0.0.1:39752
[INFO] WebSocket handshake successful in child
[INFO] Auth successful
[INFO] Command received for device
[INFO] Command received for device
```



d. command vch (web → server)



Schema Design

Users Table used stores admin credentials:

```
Create TABLE IF NOT EXISTS users (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  username TEXT UNIQUE NOT NULL,  
  password_hash TEXT NOT NULL,  
  created_at INTEGER NOT NULL  
);
```

devices Table - Persistent device registry:

```
Create TABLE IF NOT EXISTS devices(  
  device_id TEXT PRIMARY KEY,  
  device_type TEXT NOT NULL,  
  state TEXT NOT NULL DEFAULT 'offline',  
  last_seen Integer NOT NULL DEFAULT 0  
);
```

activity log table - command audit trail:

```
CREATE TABLE IF NOT EXISTS activity-log(  
  id Integer PRIMARY KEY AUTOINCREMENT,  
  timestamp INTEGER NOT NULL,  
  username TEXT NOT NULL DEFAULT "",  
  device_id TEXT NOT NULL DEFAULT "",  
  action TEXT NOT NULL,  
  source_ip TEXT NOT NULL DEFAULT ""  
);
```

Tables

activity_log

11

> devices

4

sqlite_sequence

2

users

1

devices

Refresh

Schema

Export CSV

Export JSON

Copy All

Visualize

> SQL Query Editor (click to expand)

4 total rows | 4 columns

ROWID	DEVICE_ID	DEVICE_TYPE	STATE	LAST_SEEN
4	koi_pond_lights	light	on	1774930093
11	camera_1	camera	inactive	1774947397
12	light_1	light	off	1774947397
13	fan_1	fan	off	1774947387

 Tables activity_log

11

 devices

4

>  sqlite_sequence

2

 users

1

sqlite_sequence

 Refresh Schema Export CSV Export JSON Copy All Visualize

>_ SQL Query Editor (click to expand)

 2 total rows | 2 columns

ROWID	NAME	SEQ
1	users	1
2	activity_log	13

Tables

activity_log 13

devices 4

sqlite_sequence 2

> users 1

users

Refresh

Schema

Export CSV

Export JSON

Copy All

Visualize

> SQL Query Editor (click to expand)

1 total rows | 4 columns

ID	USERNAME	PASSWORD_HASH	CREATED_AT
1	admin	240be518fabd2724ddb6f04eeb1da5967448d7e831c08c8fa822809f74c720a9	1774929933

Tables

> **activity_log** 13

devices 4

sqlite_sequence 2

users 1

activity_log

Refresh

Schema

Export CSV

Export JSON

Copy All

Visualize

> SQL Query Editor (click to expand)

13 total rows | 6 columns

ID	TIMESTAMP	USERNAME	DEVICE_ID	ACTION	SOURCE_IP
1	1774930056	admin	camera_1	active	127.0.0.1
2	1774930058	admin	fan_1	on	127.0.0.1
3	1774930096	admin	fan_1	off	127.0.0.1
4	1774930196	admin	fan_1	on	127.0.0.1
5	1774930198	admin	light_1	on	127.0.0.1
6	1774930199	admin	camera_1	inactive	127.0.0.1

Kotha Rishel Reddy 230953176 CLE-C-24
Prepared Statements

All queries are compiled once at startup as `sqlite3_stmt` pointers and reused with `sqlite3_bind * 1` `sqlite3_reset()`. This prevents SQL injection and avoids per-query compilation cost.

SQL Prepared statements

Variable	SQL Purpose
<code>stmt_user_lookup</code>	Fetch password hash by name
<code>stmt_device_insert</code>	insert or replace device row
<code>stmt_device_update</code>	update device state + last-seen
<code>stmt_device_list</code>	Fetch all devices (cursor)
<code>stmt_log_insert</code>	insert activity log row
<code>stmt_log_fetch</code>	Fetch last N log rows

Database Synchronization & Real-time Impact

Problem: Mult child processes can call `db_update_device` concurrently.

Solution: WAC mode:

`PRAGMA journal_mode = WAC;`

`PRAGMA synchronous = Normal;`

Activity log writer path - serialised via pipe:

All log writes go through a pipe to the parents `db_write_thread`, which is the only thread that calls `db_log_activity`. This eliminates writer concurrent writer contention for most frequent write path.

//_

Kotha Rushel Reddy 230453176 CCE-C-24

Real time Input Analysis

Operation	Frequency	Latency	Impact
db-log-activity	Every command	$\sim 0.2ms$	Low
db-update-device	Every heartbeat(10s)	$\sim 0.3ms$	Negligible
db-get-activity-log	Every 5s (poll)	$\sim 1ms$	Acceptable

Authoritative real-time source: The in-memory Shared Registry
NOT the database, The DB is used only for persistence and log queries.

Activity Log

Last 20 events across all devices

Time	User	Device	Action
2:29:25 PM	admin	camera_1	active
2:29:25 PM	admin	light_1	on
2:29:20 PM	admin	fan_1	on
2:29:20 PM	admin	fan_1	on
2:23:37 PM	admin	camera_1	active
9:48:22 AM	admin	light_1	on
9:46:32 AM	admin	light_1	off
9:46:28 AM	admin	camera_1	active
9:46:27 AM	admin	fan_1	off
9:46:26 AM	admin	fan_1	on
9:46:24 AM	admin	light_1	on
9:39:59 AM	admin	camera_1	inactive
9:39:58 AM	admin	light_1	on
9:39:56 AM	admin	fan_1	on
9:38:16 AM	admin	fan_1	off
9:37:38 AM	admin	fan_1	on

Tanish Sam Nathar
230953192

Authentication and TCP State Impact:

Authentication Flow:

-> Step 1 - Password hashing (SHA-256): Passwords are never stored in plain text. On login, the input password is hashed with SHA-256 using OpenSSL and compared against the stored hash:

```
char * auth_hash_password (const char * password) {  
    unsigned char hash[SHA256_DIGEST_LENGTH];  
    SHA256((const unsigned char *) password, strlen(password),  
           hash);  
}
```

Default seed: username admin, password admin123 ->
SHA-256: 2406e518fabd.....

-> Step 2 - Session Token generation: On successful login, 32 random bytes are read from /dev/urandom and base-64 encoded into a 64-character token:

```
void auth_generate_token (char * out_token) {  
    int fd = open("/dev/urandom", O_RDONLY);  
    read(fd, out_token, 32);  
}
```

-> Step 3 - Token validation on every subsequent message:

Every get_devices, command, and get_log message must include token. The server calls auth_validate(token()) before processing.

```
int auth_validate_token(const Connection* conn, const char*  
char* token) {  
    return (strcmp(conn->session_token, token) == 0) ? 1 : 0;  
}
```

3 - Attempt Lockout: After 3 consecutive failed login attempts, the server sends a final error message, closes the WebSocket connection with code 1008 (Policy Violation), and sets a close_conn flag to break the child process loop.

* WebSocket Security Design:

-> Our implementation uses plain ws:// (no TLS) as required by the project spec. In a production deployment, wss:// (WebSocket Secure over TLS) would be used - this encrypts the entire connection so that the tokens and commands cannot be intercepted.

why ws:// is acceptable:

- > Server runs only on localhost
- > Network traffic does not traverse public infrastructure
- > The SHA-256 password hash still protects credentials at rest in DB

Inspector

Console

Debugger

Network

Style Editor

Performance

Memory

Storage

Accessibility

Application

WS

8

Disable Cache

No Throttling

Status

Method

Domain

File

Initiator

Type

Transferred

Size

Headers

Cookies

Request

Response

Timings

Stack Trace

101

GET

127.0.0.1:8080

/

app.js:35 (websocket)

plain

129 B

0 B

All

Filter Messages

Data

Time

101

GET

127.0.0.1:8080

/

app.js:35 (websocket)

plain

129 B

0 B

↑

["type":"auth","username":"admin","password":"point"]

14:55:00.229

101

GET

127.0.0.1:8080

/

app.js:35 (websocket)

plain

129 B

0 B

↓

["type":"auth_result","success":false,"message":"Invalid credentials"]

14:55:00.230

101

GET

127.0.0.1:8080

/

app.js:35 (websocket)

plain

129 B

0 B

↑

["type":"auth","username":"admin","password":"iou"]

14:55:45.924

101

GET

127.0.0.1:8080

/

app.js:35 (websocket)

plain

129 B

0 B

↓

["type":"auth_result","success":false,"message":"Invalid credentials"]

14:55:45.925

101

GET

127.0.0.1:8080

/

app.js:35 (websocket)

plain

129 B

0 B

↑

["type":"auth","username":"admin","password":"kooo"]

14:55:50.686

101

GET

127.0.0.1:8080

/

app.js:35 (websocket)

plain

129 B

0 B

↓

["type":"auth_result","success":false,"message":"Too many failed attempts & closing connection"]

14:55:50.686

101

GET

127.0.0.1:8080

/

app.js:35 (websocket)

plain

129 B

0 B

Connection Closed: 1008

TCP state Analysis : TIME_WAIT and CLOSE_WAIT

→ TIME_WAIT :

- Occurs on the server side after the server actively closes a connection
- The socket stays in TIME_WAIT for $2 \times \text{MSL}$ to absorb any delayed TCP segments that arrive after the connection closes
- Impact on our system : If the server restarts without `SO_REUSEADDR`, bind() fails because port 8080 is occupied by a TIME_WAIT socket.
- Our fix : `setsockopt( sock_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt))` - allows immediate re-binding during TIME_WAIT

→ CLOSE_WAIT :

- Occurs when the remote peer (client/service) sends FIN but our server has not yet closed its end
- A large number of CLOSE_WAIT sockets = resource leak (each hold a file descriptor)
- Our fix : `SO_KEEPALIVE` is set on every accepted socket. The OS sends keepalive probes after ~ 2 hours of inactivity, if the remote is unreachable, the connection is reset, moving the socket out of CLOSE_WAIT automatically.
- Additionally, the heartbeat thread marks devices as offline after `HEARTBEAT_TIMEOUT` seconds using the in-memory registry

TCP State flow for a device disconnect:

Established $\rightarrow$ (device dies, no FIN)

↓ keep alive probes sent
↓ no response after 9 probes

CLOSE_WAIT $\rightarrow$ kernel reset

↓

CLOSED $\rightarrow$ child process

↓

don't set `offload()` called $\rightarrow$ etc

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS
deep@deep-ASUS-TUF-Gaming-A15-FA506QM-FA506QM:~/Videos/Sem-6-Labs/NPACN_Fisac
$ python3 simulator/device_sim.py --port 8080
[light 1] Heartbeat sent
^CTraceback (most recent call last):
  File "/home/deep/Videos/Sem-6-Labs/NPACN_Fisac/simulator/device_sim.py", line 98, in <module>
    main()
  File "/home/deep/Videos/Sem-6-Labs/NPACN_Fisac/simulator/device_sim.py", line 95, in main
    t.join()
  File "/usr/lib/python3.10/threading.py", line 1096, in join
    self._wait_for_tstate_lock()
  File "/usr/lib/python3.10/threading.py", line 1116, in _wait_for_tstate_lock
    if lock.acquire(block, timeout):
KeyboardInterrupt
^CException ignored in: <module 'threading' from '/usr/lib/python3.10/threading.py'>
Traceback (most recent call last):
  File "/usr/lib/python3.10/threading.py", line 1567, in _shutdown
    lock.acquire()
KeyboardInterrupt:

deep@deep-ASUS-TUF-Gaming-A15-FA506QM-FA506QM:~/Videos/Sem-6-Labs/NPACN_Fisac$ python3 simulator/de
vice_sim.py --port 8080
[fan 1] Connected to ws://localhost:8080
[fan 1] Registration result: {"type": "register_ack", "success": true, "device_id": "fan_1"}
[camera 1] Connected to ws://localhost:8080
[light 1] Connected to ws://localhost:8080
[light 1] Registration result: {"type": "register_ack", "success": true, "device_id": "light_1"}
[camera 1] Registration result: {"type": "register_ack", "success": true, "device_id": "camera_1"}
[fan 1] Received command: on
[camera 1] Received command: active
[light 1] Received command: on
[fan 1] Heartbeat sent
[camera 1] Heartbeat sent
[light 1] Heartbeat sent
[camera 1] Heartbeat sent
[fan 1] Heartbeat sent
[light 1] Heartbeat sent
^CTraceback (most recent call last):
  File "/home/deep/Videos/Sem-6-Labs/NPACN_Fisac/simulator/device_sim.py", line 98, in <module>
    main()
  File "/home/deep/Videos/Sem-6-Labs/NPACN_Fisac/simulator/device_sim.py", line 95, in main
    t.join()
  File "/usr/lib/python3.10/threading.py", line 1096, in join
    self._wait_for_tstate_lock()
  File "/usr/lib/python3.10/threading.py", line 1116, in _wait_for_tstate_lock
    if lock.acquire(block, timeout):
KeyboardInterrupt
^CException ignored in: <module 'threading' from '/usr/lib/python3.10/threading.py'>
Traceback (most recent call last):
  File "/usr/lib/python3.10/threading.py", line 1567, in _shutdown
    lock.acquire()
KeyboardInterrupt:

deep@deep-ASUS-TUF-Gaming-A15-FA506QM-FA
deep@deep-ASUS-TUF-Gaming-A15-FA506QM-FA
deep@deep-ASUS-TUF-Gaming-A15-FA506QM-FA506QM:~/Videos/Sem-6-Labs/NPACN_Fisac
```

← → ↺

http://127.0.0.1:3000

☆ Sign in

SmartHome

Devices

Monitor and control your connected devices

Add Device

Devices

Activity Log

3

TOTAL

0

ONLINE

3

OFFLINE

FAN

fan_1

offline

Offline

CAMERA

camera_1

offline

Offline

LIGHT

light_1

offline

Offline

Live

Sign Out

Devices

Monitor and control your connected devices

Add Device

Devices

Activity Log

3

TOTAL

0

ONLINE

3

OFFLINE

FAN

fan_1

offline

Offline

CAMERA

camera_1

offline

Offline

LIGHT

light_1

offline

Offline

Live

Sign Out

Inspector Console Debugger Network Style Editor Performance Memory Storage Accessibility Application

WS

Status	Method	Domain	File	Initiator	Type	Transferred	Size
101	GET	127.0.0.1:8080	/	app.js:35 (websocket)	plain	129 B	0 B

All HTML CSS JS XHR Fonts Images Media WS Other Disable Cache No Throttling

Headers Cookies Request Response Timings Stack Trace

All Filter Messages

Data	Time
↓ ("type":"activity_log","entries":[{"timestamp":1774949256,"username":"admin","device_id":"lig...	14:58:12.471
↑ ("type":"get_devices","token":"8c16f8366dc90c031fff84c878641df329c8288caa8d3d4d63298b...	14:58:13.468
↓ ("type":"device_list","devices":[{"device_id":"fan_1","device_type":"fan","state":"offline",last_s...	14:58:13.469
↑ ("type":"get_devices","token":"8c16f8366dc90c031fff84c878641df329c8288caa8d3d4d63298b...	14:58:16.469
↓ ("type":"device_list","devices":[{"device_id":"fan_1","device_type":"fan","state":"offline",last_s...	14:58:16.469
↑ ("type":"get_log","token":"8c16f8366dc90c031fff84c878641df329c8288caa8d3d4d63298bec72...	14:58:17.471
↓ ("type":"activity_log","entries":[{"timestamp":1774949256,"username":"admin","device_id":"lig...	14:58:17.471
↑ ("type":"get_devices","token":"8c16f8366dc90c031fff84c878641df329c8288caa8d3d4d63298b...	14:58:19.490
↓ ("type":"device_list","devices":[{"device_id":"fan_1","device_type":"fan","state":"offline",last_s...	14:58:19.490
↑ ("type":"get_log","token":"8c16f8366dc90c031fff84c878641df329c8288caa8d3d4d63298bec72...	14:58:22.472
↓ ("type":"activity_log","entries":[{"timestamp":1774949256,"username":"admin","device_id":"lig...	14:58:22.472
↑ ("type":"get_devices","token":"8c16f8366dc90c031fff84c878641df329c8288caa8d3d4d63298b...	14:58:22.490

Devices

Monitor and control your connected devices

Add Device

Devices

Activity Log

3

TOTAL

3

ONLINE

0

OFFLINE

FAN

fan_1

on

Toggle State

CAMERA

camera_1

active

Toggle State

LIGHT

light_1

on

Toggle State

Live

Sign Out

Inspector Console Debugger Network Style Editor Performance Memory Storage Accessibility Application

WS

Status	Method	Domain	File	Initiator	Type	Transferred	Size
101	GET	127.0.0.1:8080	/	app.js:35 (websocket)	plain	129 B	0 B

All HTML CSS JS XHR Fonts Images Media WS Other Disable Cache No Throttling

Headers Cookies Request Response Timings Stack Trace

All Filter Messages

Data	Time
↓ ("type":"command_ack","success":true,"device_id":"camera_1","action":"active")	14:58:32.937
↑ ("type":"get_devices","token":"8c16f8366dc90c031fff84c878641df329c8288caa8d3d4d63298b...	14:58:32.938
↑ ("type":"get_log","token":"8c16f8366dc90c031fff84c878641df329c8288caa8d3d4d63298bec72...	14:58:32.938
↓ ("type":"device_list","devices":[{"device_id":"fan_1","device_type":"fan","state":"on","last_seen"...	14:58:32.938
↓ ("type":"activity_log","entries":[{"timestamp":1774949312,"username":"admin","device_id":"ca...	14:58:32.938
↑ ("type":"command","token":"8c16f8366dc90c031fff84c878641df329c8288caa8d3d4d63298bec...	14:58:33.631
↓ ("type":"command_ack","success":true,"device_id":"light_1","action":"on")	14:58:33.631
↑ ("type":"get_devices","token":"8c16f8366dc90c031fff84c878641df329c8288caa8d3d4d63298b...	14:58:33.632
↑ ("type":"get_log","token":"8c16f8366dc90c031fff84c878641df329c8288caa8d3d4d63298bec72...	14:58:33.632
↓ ("type":"device_list","devices":[{"device_id":"fan_1","device_type":"fan","state":"on","last_seen"...	14:58:33.632
↓ ("type":"activity_log","entries":[{"timestamp":1774949313,"username":"admin","device_id":"lig...	14:58:33.632
↑ ("type":"get_devices","token":"8c16f8366dc90c031fff84c878641df329c8288caa8d3d4d63298b...	14:58:34.499